

Data Warehousing & Caching

Vũ Tuyết Trinh

1

Data Warehousing and Materialization

- We have mostly focused on techniques for *virtual* data integration (see Ch. 1)
 - Queries are composed with mappings **on the fly** and data is fetched on demand
 - This represents one extreme point
- In this chapter, we consider cases where data is transformed and materialized “in advance” of the queries
 - The main scenario: the **data warehouse**

2

What Is a Data Warehouse?

- In many organizations, we want a central “store” of all of our entities, concepts, metadata, and historical information
 - For doing data validation, complex mining, analysis, prediction, ...
 - This is the data warehouse
- To this point we’ve focused on scenarios where the data “lives” in the sources – here we may have a “master” version (and archival version) in a central database
 - For performance reasons, availability reasons, archival reasons, ...

In the Rest of this Chapter...

- The data warehouse as the **master data** instance
- Data warehouse architectures, design, loading
- Data exchange: declarative data warehousing
- Hybrid models: caching and partial materialization
- Querying externally archived data

Outline

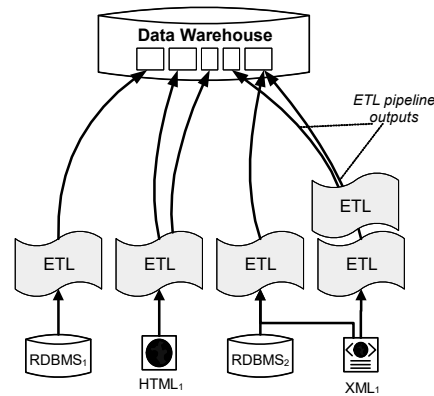
- The data warehouse
 - Motivation: Master data management
 - Physical design
 - Extract/transform/load
- Data exchange
- Caching & partial materialization
- Operating on external data

Master Data Management

- One of the “modern” uses of the data warehouse is not only to support **analytics** but to serve as a reference to all of the entities in the organization
 - A cleaned, validated repository of what we know
 - ... which can be linked to by data sources
 - ... which may help with data cleaning
 - ... and which may be the basis of **data governance** (processes by which data is created and modified in a systematic way, e.g., to comply with gov’t regulations)
- There is an emerging field called **master data management** out the process of creating these

Data Warehouse Architecture

- At the top – a centralized database
 - Generally configured for queries and appends – not transactions
 - Many indices, materialized views, etc.
- Data is loaded and periodically updated via **Extract/Transform/Load (ETL) tools**

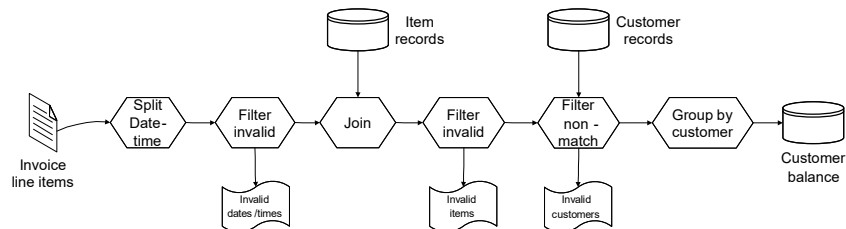


ETL Tools

- ETL tools are the equivalent of **schema mappings** in virtual integration, but are more powerful
- Arbitrary pieces of code to take data from a source, convert it into data for the warehouse:
 - **import filters** – read and convert from data sources
 - **data transformations** – join, aggregate, filter, convert data
 - **de-duplication** – finds multiple records referring to the same entity, merges them
 - **profiling** – builds tables, histograms, etc. to summarize data
 - **quality management** – test against master values, known business rules, constraints, etc.

Example ETL Tool Chain

- This is an example for e-commerce loading
- Note multiple stages of filtering (using selection or join-like operations), logging bad records, before we group and load



Basic Data Warehouse – Summary

- Two aspects:
 - A central DBMS optimized for appends and querying
 - The “master data” instance
 - Or the instance for doing mining, analytics, and prediction
 - A set of procedural ETL “pipelines” to fetch, transform, filter, clean, and load data
 - Often these tools are more expressive than standard conjunctive queries
 - ... But not always!

This raises a question – can we do warehousing with **declarative** mappings?

Datawarehouse OLAP



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

11

Outline

- ✓ The data warehouse
- Data exchange
 - Caching & partial materialization
 - Operating on external data



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

12

Data Exchange

- Intuitively, a **declarative** setup for data warehousing
 - Declarative schema mappings
 - Materialized database as in the previous section
- Also allow for **unknown** values when we map from source to target (warehouse) instance
 - If we know a professor teaches a student, then there must exist a course C that the student took and the professor taught – but we may not know which...



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

13

Data Exchange Formulation

A data exchange **setting** (S, T, M, C_T) has:

- **S**, source schema representing *all* of the source tables jointly
- **T**, target schema
- A set of mappings or tuple-generating dependencies

$$(\forall \bar{X}) s_1(\bar{X}_1), \dots, s_m(\bar{X}_m) \rightarrow (\exists \bar{Y}) t_1(\bar{Y}_1), \dots, t_k(\bar{Y}_k)$$

relating S and T

- A set of constraints (equality-generating dependencies)

$$(\exists \bar{Y}) t_1(\bar{Y}_1), \dots, t_l(\bar{Y}_l) \rightarrow (Y_i = Y_j)$$



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

14

An Example

Source S has

Teaches(*prof*, *student*)
Adviser(*adviser*, *student*)

Target T has

Advise(*adviser*, *student*)
TeachesCourse(*prof*, *course*)
Takes(*course*, *student*)

$r_1 : \text{Teaches}(\text{prof}, \text{stud}) \rightarrow \exists D. \text{Advise}(D, \text{stud})$

$r_2 : \text{Teaches}(\text{prof}, \text{stud}) \rightarrow \exists C. \text{TeachesCourse}(\text{prof}, C), \text{Takes}(C, \text{stud})$

$r_3 : \text{Adviser}(\text{prof}, \text{stud}) \rightarrow \text{Advise}(\text{prof}, \text{stud})$

$r_4 : \text{Adviser}(\text{prof}, \text{stud}) \rightarrow \exists C, D. \text{TeachesCourse}(D, C), \text{Takes}(C, \text{stud})$

↑
existential variables represent unknowns

The Data Exchange Solution

- The goal of data exchange is to compute an instance of the target schema, given a data exchange setting $D = (S, T, M, C_T)$ and an instance $I(S)$
- An instance J of Schema T is a data exchange solution for D and I if
 1. the pair (I, J) satisfies schema mapping M , and
 2. J satisfies constraints C_T

Back to the Example, Now with Data

Instance I(S) has

Teaches

prof	student
Ann	Bob
Chloe	David

Adviser

adviser	student
Ellen	Bob
Felicia	David

Instance J(T) has

Advise

adviser	student
Ellen	Bob
Felicia	David

TeachesCourse

prof	course
Ann	C_1
Chloe	C_2

Takes

course	student
C_1	Bob
C_2	David

*variables or labeled nulls
represent unknown values*



TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

17

Back to the Example, Now with Data

Instance I(S) has

Teaches

prof	student
Ann	Bob
Chloe	David

Adviser

adviser	student
Ellen	Bob
Felicia	David

Instance J(T) has

Advise

adviser	student
Ellen	Bob
Felicia	David

TeachesCourse

prof	course
Ann	C_1
Chloe	C_2

Takes

course	student
C_1	Bob
C_2	David

*this time the labeled
nulls are all the same!*



TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

18

Universal Solutions

- Intuitively, the first solution should be better than the second
 - The second solution uses the same variable for the course taught by Ann and by Chloe – they are the same course
 - But this was not specified in the original schema!
- We formalize that through the notion of the **universal** solution, which must not **lose** any information



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

19

Formalizing the Universal Solution

First we define **instance homomorphism**:

- Let J_1, J_2 be two instances of schema T
- A mapping $h: J_1 \rightarrow J_2$ is a *homomorphism* from J_1 to J_2 if
 - $h(c) = c$ for every $c \in C$,
 - for every tuple $R(a_1, \dots, a_n) \in J_1$ the tuple $R(h(a_1), \dots, h(a_n)) \in J_2$
- J_1, J_2 are *homomorphically equivalent* if there are homomorphisms $h: J_1 \rightarrow J_2$ and $h': J_2 \rightarrow J_1$

Def: Universal solution for data exchange setting $D = (S, T, M, C_T)$, where I is an instance of S .

A data exchange solution J for D and I is a *universal solution* if, for every other data exchange solution J' for D and I , there exists a homomorphism $h: J \rightarrow J'$



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

20

Computing Universal Solutions

- The standard process is to use a procedure called the *chase*
- Informally:
 - Consider every formula r of M in turn:
 - If there is a variable substitution for the left-hand side (lhs) of r where the right-hand side (rhs) is not in the solution – add it
 - If we create a new tuple, for every existential variable in the rhs, substitute a new fresh variable



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

21

Core Universal Solutions

- Universal solutions may be of arbitrary size
- The **core universal solution** is the minimal universal solution

$$J_m = \{\text{TeachesCourse}(\text{Ann}, C_1), \text{Takes}(C_1, \text{Bob})$$
$$\quad \text{TeachesCourse}(\text{Chloe}, C_2), \text{Takes}(C_2, \text{David})$$
$$\quad \dots$$
$$\quad \text{TeachesCourse}(\text{Ann}, C_{2m-1}), \text{Takes}(C_{2m-1}, \text{Bob})$$
$$\quad \text{TeachesCourse}(\text{Chloe}, C_{2m}), \text{Takes}(C_{2m}, \text{David})$$
$$\quad \text{Advise}(\text{Ellen}, \text{Bob}), \text{Advise}(\text{Felicia}, \text{David})\}$$


SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

22

Data Exchange and Querying

- As with the data warehouse, all queries are directly posed over the target database – no reformulation necessary
- However, we typically assume **certain answers** semantics
 - To get the certain answers (which are the same as in the virtual integration setting with GLAV/TGD mappings) – compute the query answers and then **drop** any tuples with labeled nulls (variables)

Data Exchange vs. Warehousing

- From an external perspective, exchange and warehousing are essentially equivalent
- But there are different trade-offs in procedural vs. declarative mappings
 - Procedural – more expressive
 - Declarative – easier to reason about, compose, invert, create materialized views for, etc.

Outline

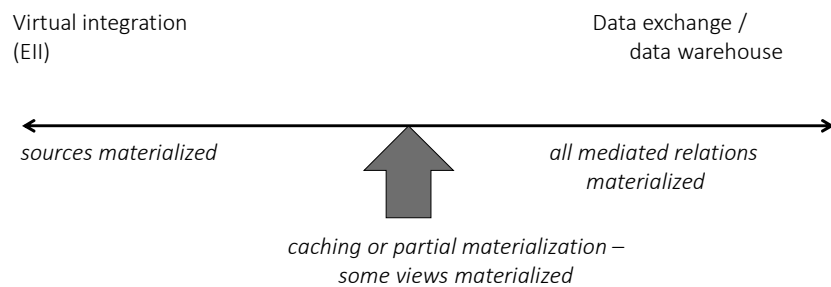
- ✓ The data warehouse
- ✓ Data exchange
- Caching & partial materialization
- Operating on external data



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

25

The Spectrum of Materialization



- Many real EII systems compute and maintain materialized views, or cache results
A “hybrid” point between the fully virtual and fully materialized approaches



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

26

Possible Techniques for Choosing What to Materialize

Cache results of prior queries

- Take the results of each query, materialize them
- Use answering queries using views to reuse
- Expire using time-to-live... May not always be fresh!

Administrator-selected views

- Someone manually specifies views to compute and maintain, as with a relational DBMS
- System automatically maintains

Automatic view selection

- Using query workload, update frequencies – a view materialization wizard chooses what to materialize



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

27

Outline

- ✓The data warehouse
- ✓Data exchange
- ✓Caching & partial materialization
- Operating on external data



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

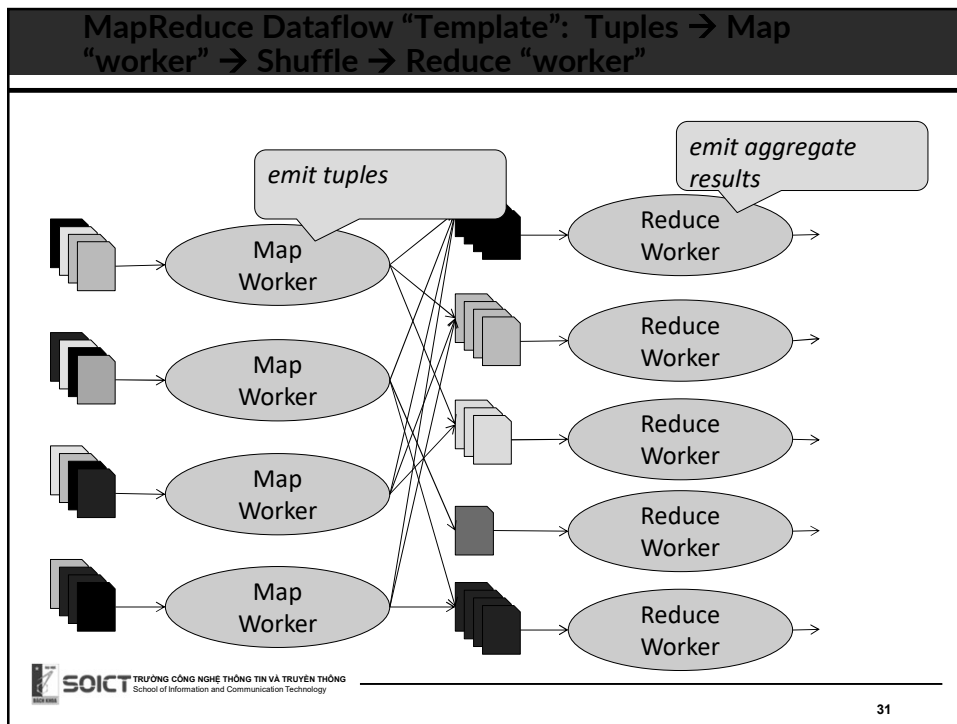
28

Many “Integration-Like” Scenarios over Historical Data

- Many Web scenarios where we have large logs of data accesses, created by the server
- Goal: put these together and query them!
- Looks like a very simple data integration scenario – external data, but single schema
- A common approach: use programming environments like MapReduce (or SQL layers above) to query the data on a cluster
 - MapReduce reliably runs large jobs across 100s or 1000s of “shared nothing” nodes in a cluster

MapReduce Basics

- MapReduce is essentially a template for writing distributed programs – corresponding to a single SQL SELECT..FROM..WHERE..GROUP BY..HAVING block with *user-defined functions*
- The MapReduce runtime calls a set of functions:
 - **map** is given a tuple, outputs 0 or more tuples in response
 - roughly like the WHERE clause
 - **shuffle** is a stage for doing sort-based grouping on a key (specified by the map)
 - **reduce** is an aggregate function called over the set of tuples with the same grouping key



31

MapReduce as ETL

- Some people use MapReduce to take data, transform it, and load it into a warehouse
- ... which is basically what ETL tools do!
- The dividing line between DBMSs, EII, MapReduce is blurring as of the development of this book
 - SQL → MapReduce
 - MapReduce over SQL engines
 - Shared-nothing DBMSs
 - NoSQL

SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

32

Warehousing & Materialization Wrap-up

- There are benefits to centralizing & materializing data
 - Performance, especially for analytics / mining
 - Archival
 - Standardization / canonicalization
- Data warehouses typically use procedural ETL tools to extract, transform, load (and clean) data
- Data exchange replaces ETL with declarative mappings (where feasible)
- Hybrid schemes exist for partial materialization
- Increasingly we are integrating via MapReduce and its cousins